

Deep Learning

Homework 1

Introduction, Optimization, and Convolutional Neural Networks

Total Points: 25

Instructions

This assignment is worth a total of 25 points. You may use either PyTorch or TensorFlow/Keras. Use the MNIST dataset for Problems 1 and 2, and the CIFAR-10 dataset for Problem 3.

Your submission should include a Jupyter notebook (`.ipynb`) or Python scripts (`.py`) containing your code, together with a PDF report presenting your results, plots, and a brief analysis. Your code should be well commented and fully reproducible, and you should include a `requirements.txt` file listing all dependencies.

Problem 1: Implementing and Training MLPs (8 points)

In this problem, you will implement a multilayer perceptron from scratch and compare different activation functions.

Part A: MLP Implementation (3 points)

Implement a flexible MLP class that supports an arbitrary number of hidden layers and neurons, different activation functions (ReLU, Sigmoid, and Tanh), and forward pass computation. Train your MLP on the MNIST digit classification dataset using the following architecture.

Input	784 (28×28 flattened images)
Hidden Layer 1	128 neurons
Hidden Layer 2	64 neurons
Output	10 classes (softmax)

Deliverables. Submit a working MLP implementation and report both the training accuracy and the test accuracy.

Part B: Activation Function Comparison (3 points)

Train three models using the same architecture but different activation functions: ReLU, Sigmoid, and Tanh.

Deliverables. Plot the training loss curves for all three models on the same graph and provide a table comparing the final test accuracy for each activation function. Include a brief analysis (3–4 sentences) explaining which activation function performed best and why.

Part C: Solving XOR (2 points)

Implement and train a small MLP to solve the XOR problem. The network should take 2 input features, use a hidden layer of 2–4 neurons (your choice), and produce a single output neuron with sigmoid activation.

Deliverables. Submit the code for the XOR MLP, a visualization of the decision boundary learned by your network, and the final accuracy on all four XOR cases.

Problem 2: Optimization and Training Dynamics (9 points)

This problem explores different optimization algorithms and training techniques.

Part A: Optimizer Comparison (4 points)

Train the same MLP architecture from Problem 1A using each of the four optimizers below, training each model for 20 epochs on MNIST.

SGD	Learning rate = 0.01
SGD with Momentum	Momentum = 0.9, learning rate = 0.01
RMSprop	Learning rate = 0.001
Adam	Learning rate = 0.001

Deliverables. Plot the training loss curves and the test accuracy curves for all four optimizers, with each metric shown on a single shared graph. Provide a table listing the final test accuracy and training time for each optimizer, and write a brief analysis (4–5 sentences) comparing their convergence speed and final performance.

Part B: Gradient Analysis (3 points)

Implement a deep network with five or more hidden layers and use it to investigate the vanishing gradient problem. First train the deep MLP with sigmoid activations while tracking and plotting the gradient magnitudes at different layers during training, then repeat the experiment using ReLU activations.

Deliverables. Submit code showing your gradient tracking implementation and a plot comparing gradient magnitudes across layers for sigmoid versus ReLU. Include a brief explanation (3–4 sentences) of what you observe about the vanishing gradient problem.

Part C: Regularization Effects (2 points)

Train models using three configurations: a baseline with no regularization, L2 regularization with a weight decay of 0.001, and dropout with $p = 0.5$ applied to the hidden layers.

Deliverables. Provide a plot showing training versus test accuracy for each configuration, and analyze in 2–3 sentences which method best prevents overfitting.

Problem 3: Convolutional Neural Networks (8 points)

Implement and experiment with convolutional neural networks on the CIFAR-10 dataset.

Part A: Basic CNN Implementation (4 points)

Implement a CNN with the following architecture and train it for 20 epochs on CIFAR-10.

Conv Layer 1	32 filters, 3×3 kernel, ReLU, same padding
Pooling	MaxPool 2×2
Conv Layer 2	64 filters, 3×3 kernel, ReLU, same padding
Pooling	MaxPool 2×2
Flatten	–
Fully Connected	128 neurons, ReLU
Output	10 classes (softmax)

Deliverables. Submit the CNN implementation code, training and validation loss curves, the final test accuracy, and a confusion matrix on the test set.

Part B: Architecture Experimentation (3 points)

Design and compare three different CNN architectures by varying the number of convolutional layers, the number of filters, the kernel sizes, and the pooling strategy (max versus average pooling).

Deliverables. Provide a description of each architecture (a table is acceptable) and a comparison table showing the number of parameters, training time, and test accuracy for each. Include an analysis (4–5 sentences) discussing the trade-offs between model complexity and performance.

Part C: Visualizing Learned Features (1 point)

Visualize what your CNN has learned.

Deliverables. Visualize at least eight filters (kernels) from the first convolutional layer, and show the activation maps from the first convolutional layer for two to three sample images.

Bonus: Transfer Learning (2 points, Optional)

Use a pre-trained model such as ResNet18 or VGG16 and fine-tune it on CIFAR-10. Load weights pre-trained on ImageNet, replace the final layer with one suited to the 10 CIFAR-10 classes, and fine-tune the model.

Deliverables. Submit the code for your transfer learning implementation, compare its test accuracy with your CNN from Problem 3A, and include a brief discussion (2–3 sentences) on why transfer learning helps.